

Git Assignment 2

Merge Conflict Resolution Challenge

Integrating three teams' changes to the customer profile module safely

Field	Details
Student name	Mohit
Institution	National Institute of Information Technology
Course	Java Full stack
Instructor	Mr. Praveen
Submission date	October, 01, 2026

Table of Contents

Table of Contents	2
1. Executive Summary	3
2. Business Scenario and Objectives	3
2.1 Scenario.....	3
2.2 Objectives	3
3. Stakeholders.....	3
4. Current Challenges and Risks	4
5. Assumptions and Constraints	4
5.1 Given Requirements (from the assignment)	4
5.2 Assumptions (made by the author)	4
5.3 Constraints.....	5
6. Proposed Solution (Step-by-Step Approach).....	5
6.1 Identify the Conflicting Files.....	6
6.2 Analyse Code Ownership	6
6.3 Resolve the Conflicts	7
6.4 Test the Integrated Solution	9
6.5 Document Decisions.....	10
6.6 Why This Approach Minimises Risk.....	10
7. Implementation Plan.....	10
7.1 Integration Plan.....	10
7.2 Prevention Plan	11
8. Measurable Success Criteria	11
9. Recommendations and Lessons Learned	11
9.1 Recommendations.....	11
9.2 Lessons Learned	12

1. Executive Summary

Problem. Three teams change the same customer profile module at the same time: Team A updates validations, Team B modifies UI fields, and Team C changes database mappings. When their work is combined, conflicting changes appear.

Solution. Combine the work in a controlled order on a dedicated integration branch: data layer first (Team C), then validation (Team A), then UI (Team B). Conflicting files are found with Git, the owner of each file helps decide the fix, the result is tested after every step, and each decision is recorded.

Why this minimises risk. No change is silently overwritten, hidden "clean-merge but broken" problems are caught by testing, customer data mappings are settled first because they are hardest to undo, and every decision can be explained later to an auditor.

2. Business Scenario and Objectives

2.1 Scenario

The customer profile module stores and shows personal details of bank customers. Three teams edit it in parallel on separate branches. Each team's change is correct on its own, but together they touch the same files and the same fields. Because this is customer data in a bank, a wrong resolution can corrupt records or expose data, not just break a screen.

2.2 Objectives

#	Objective	Why it matters
O1	Find every conflicting file	A missed conflict becomes a production bug
O2	Understand who owns each changed piece of code	The owner knows what the code must do
O3	Resolve conflicts without losing any team's work	Each team's change was a valid requirement
O4	Prove the combined code works through testing	Git can merge cleanly and still produce broken code
O5	Record every decision	Needed for audit, learning and repeat conflicts
O6	Use an approach that keeps risk low	The assignment asks for this to be justified

3. Stakeholders

A stakeholder is anyone affected by, or responsible for, the outcome of the integration.

Stakeholder	Role	What they need
Team A (validation)	Updates validation rules	Rules still enforced and consistent with the database
Team B (UI)	Modifies UI fields	Screens work and match the agreed field names
Team C (database)	Changes database mappings	Data stays correct; existing records are safe
Team leads	Own each team's delivery	A fair, quick, transparent resolution
Integration lead (tech lead)	Coordinates integration, breaks ties	Clear process and decision authority
QA / Testers	Verify the combined result	A stable build with a clear list of what changed
Security and Compliance	Protect customer data, support audits	No personal data exposed; decisions on record
DevOps	Run build and deploy pipelines	A branch that builds and deploys reliably

Stakeholder	Role	What they need
Product owner	Owens features and dates	All three features delivered on time
Bank customers	Use the profile screens	Correct, secure personal data

4. Current Challenges and Risks

#	Challenge	Impact	Risk
C1	Three teams edit the same files (text conflicts)	Git stops the merge; time lost; wrong lines may be kept	High
C2	Semantic conflicts: Git merges cleanly but the code no longer works together	Runtime failures that nobody sees until testing or production	High
C3	Unclear ownership of shared files	Arguments, or the wrong team decides	Medium
C4	Quick "accept mine" or "accept theirs" resolutions	One team's work is silently lost	High
C5	Database mapping mismatch between layers	Data saved in wrong columns or lost; privacy issue	High
C6	Conflicts found only at the end	Large, hard-to-resolve conflicts	Medium
C7	Conflicting database migrations (same order number or changes)	Database upgrade fails or applies in the wrong order	High
C8	No record of why a choice was made	Same conflict repeats; audit questions unanswered	Medium
C9	Merged result not tested as a whole	Each team's tests pass but the combination fails	High

Risk level is a judgement based on the likely effect on a bank (customer data, money, reputation). It is not measured data.

5. Assumptions and Constraints

This section separates what the assignment **requires** from what this report **assumes**, so the two are never confused.

5.1 Given Requirements (from the assignment)

#	Requirement
R1	Three teams work on the same customer profile module
R2	Team A updates validations; Team B modifies UI fields; Team C changes database mappings
R3	Conflicting changes appear during integration
R4	Identify the conflicting files
R5	Analyse code ownership
R6	Resolve the conflicts
R7	Test the integrated solution
R8	Document decisions
R9	Explain why the chosen conflict resolution approach minimises risk
R10	Deliver a report, workflow diagrams, recommendations, lessons learned and an implementation plan

5.2 Assumptions (made by the author)

#	Assumption	Effect if wrong
A1	The scenario gives no file names, so the files, field names and code in this report are illustrative examples	Real file names and conflicts will differ; the method stays the same
A2	This is the same banking platform as Assignment 1 (the module is its customer-management module), so feature branches and merge requests are already used	Branch names and rules would need adjusting
A3	GitLab hosts the code and runs the automated pipeline	Menu names change; the process does not
A4	Each team has about 4 to 6 developers and a named owner	Ownership rules would need adjusting
A5	The teams branched from <code>develop</code> at different times and have not merged each other's work yet	Conflicts could be fewer or more
A6	An integration lead (tech lead) exists to break ties	A team lead rota would take this role
A7	An automated test suite exists (unit and integration tests)	More manual testing would be needed
A8	Targets in Section 8 are illustrative because no baseline data is given	Re-base targets on real data

5.3 Constraints

- **Personal data:** customer details are sensitive; resolutions must not weaken privacy or data accuracy.
- **Auditability:** a bank must be able to show who decided what, and why.
- **Parallel teams:** three teams are working at once, so coordination time is limited.
- **Irreversibility:** database changes on live data are harder to undo than code changes.
- **Scope:** this is a design and planning exercise; no real systems are changed.

6. Proposed Solution (Step-by-Step Approach)

The solution has six steps, taken in order. Each step answers one task in the assignment.

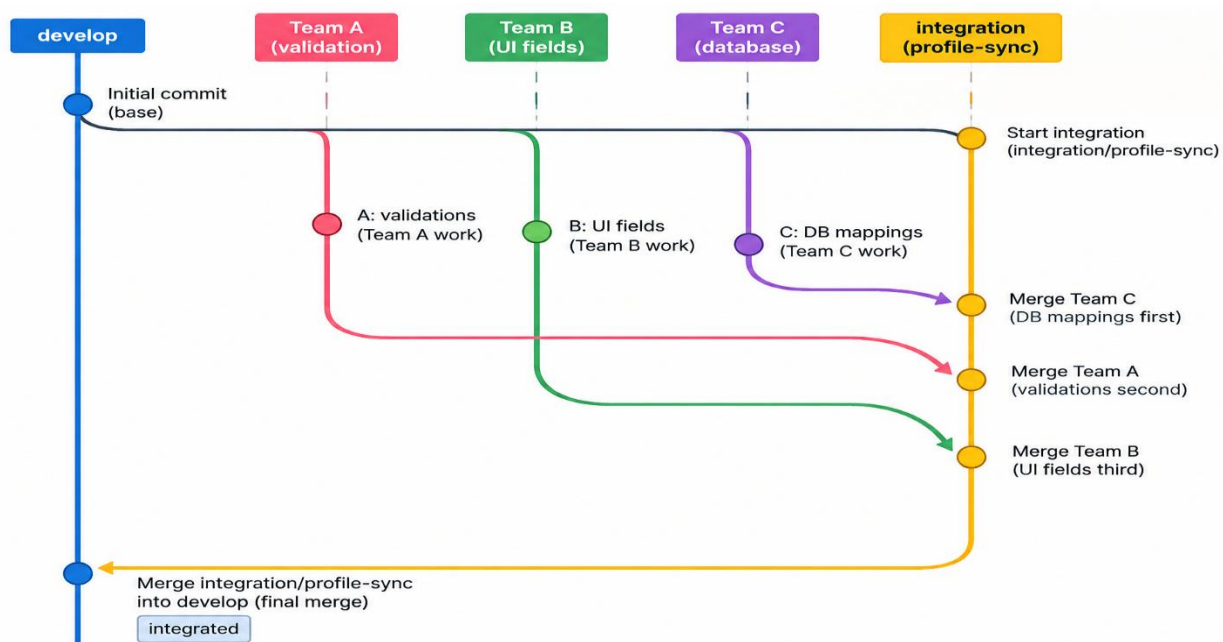


Diagram 1: THREE-TEAM CONFLICT SCENARIO

6.1 Identify the Conflicting Files

Simple explanation: a conflict happens when two branches change the same lines of the same file and Git cannot decide which to keep. There is a second, sneakier kind: Git merges without complaint but the combined code is wrong. This report calls these **text conflicts** and **semantic conflicts**.

Example (illustrative). All three teams touch the phone number field of a customer:

File (illustrative)	Team A	Team B	Team C	Conflict type
profile/api.py	Adds validation call	Renames request field	Changes DB save call	Text (A, B, C)
profile/models.py	Adds length rule	No change	Renames column mapping	Text (A, C)
ui/ProfileForm.jsx	Adds error message display	Renames field and label	No change	Text (A, B)
db/migrations/0007_*.sql	Adds a migration	No change	Adds a migration	Text: same order number (A, C)
tests/test_profile.py	Adds tests	Adds tests	Adds tests	Text: all append at the end
profile/validation.py	Checks field phone	UI now sends mobile_number	DB column is mobile_no	Semantic: no Git conflict, but the code breaks

How to find them with Git

```
git fetch origin
git checkout -b integration/profile-sync origin/develop

# Preview before merging: which files did each team change?
git diff --name-only develop...team-a/validation
git diff --name-only develop...team-b/ui-fields
git diff --name-only develop...team-c/db-mapping
# Files appearing in 2 or more lists are conflict candidates

# After a merge stops with conflicts:
git status # shows 'both modified'
git diff --name-only --diff-filter=U # lists only conflicted files
```

Semantic conflicts are not listed by Git. They are found by reading each team's change description and by testing (Section 6.4).

6.2 Analyse Code Ownership

Simple explanation: the owner of a piece of code is the person or team who understands what it must do. They should help decide how a conflict is resolved, because only they know which version is correct.

Ownership is checked in three ways:

- **CODEOWNERS file:** who is officially responsible for each folder.
- **Git history:** who wrote and changed the lines most recently.
- **Requirement owner:** which team's feature caused the change.

```
git log --format='%an' -- profile/models.py | sort | uniq -c | sort -rn # who changes it most
git blame -L 40,60 profile/models.py # who wrote these lines
git log -p --follow -- profile/api.py # history of the file
```

Ownership and decision rules

Area	Owner	Is the source of truth for	Decision rule
Database mapping and migrations	Team C	Field names, types, lengths, data safety	Data integrity comes first; Security is consulted for personal data
Validation rules	Team A	What input is acceptable	Must fit within what the database can store
UI fields and labels	Team B	What the user sees	Must follow the agreed field names; labels are Team B's choice
Shared files (API, tests)	All three	How the layers connect	Owners decide together; unresolved goes to the integration lead

Why data first? Changing a screen or a rule later is easy. Changing how stored customer data is mapped is hard to undo. So the data layer is settled first and the other layers fit around it.

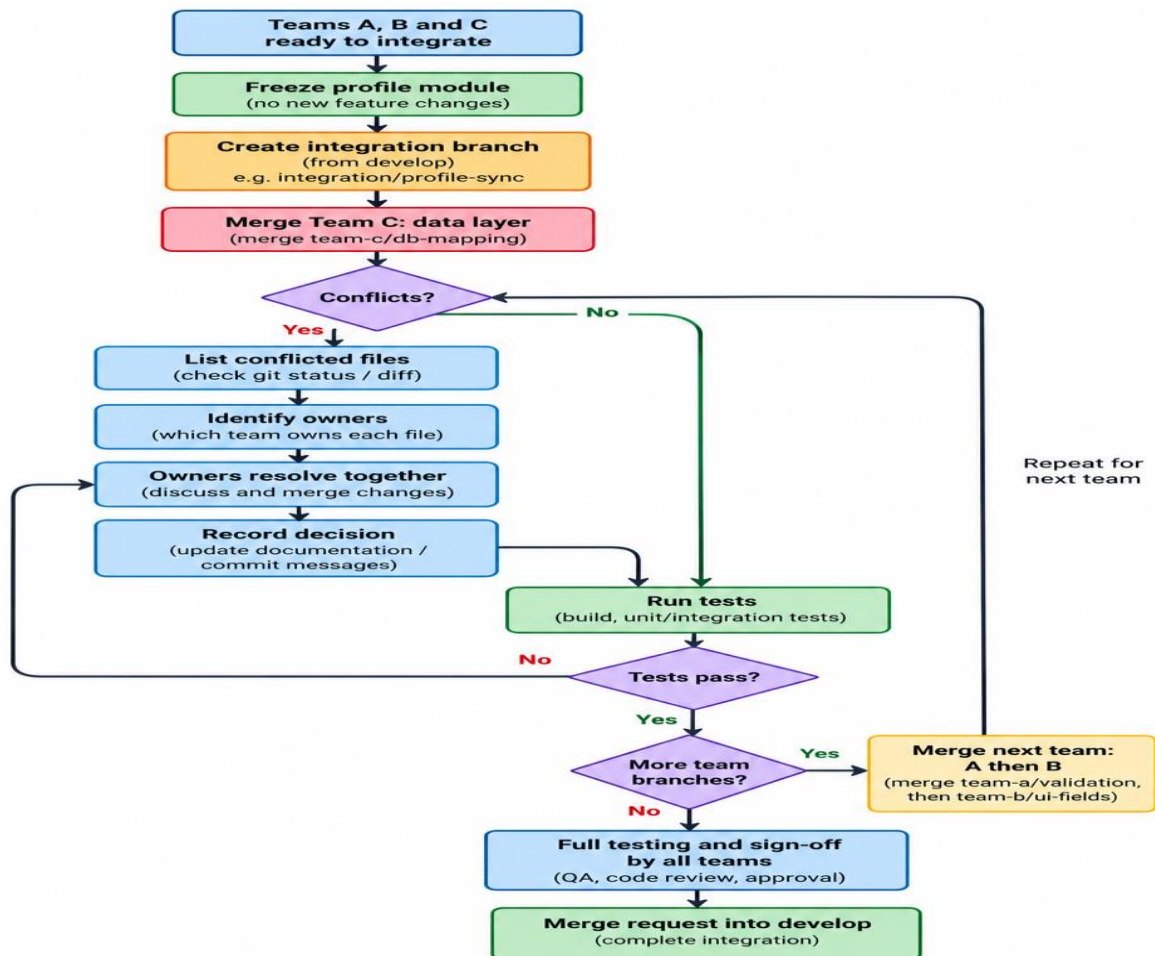


Diagram 1: CONFLICT RESOLUTION WORKFLOW

6.3 Resolve the Conflicts

Options considered

Approach	What it means	Risk	Verdict
Accept one side blindly	Use "ours" or "theirs" for the whole file	Silently loses a team's work	Rejected
One person resolves everything	A single developer fixes all conflicts alone	Bottleneck; lacks knowledge of other layers	Rejected

Approach	What it means	Risk	Verdict
Each team resolves on its own branch	Teams rebase separately	Different fixes for the same issue; late surprises	Rejected
Sequential integration with owners and tests	Merge one team at a time on an integration branch; owners resolve together; test after each merge; record decisions	Lowest: small steps, expert decisions, early detection	Chosen

Resolution procedure

1. **Freeze:** no new changes to the profile module until integration finishes.
2. **Create** `integration/profile-sync` from `develop`.
3. **Merge Team C first** (data layer), then **Team A** (validation), then **Team B** (UI). One at a time.
4. **On conflict:** list the conflicted files, identify the owners, and hold a short "conflict huddle" with those owners (aim for 30 minutes).
5. **Resolve** by combining both intentions, not by deleting one. Remove all markers.
6. **Run the tests** after each merge before moving to the next team.
7. **Record** each decision (Section 6.5).
8. **Merge request** the finished integration branch into `develop`, with sign-off from all three teams.

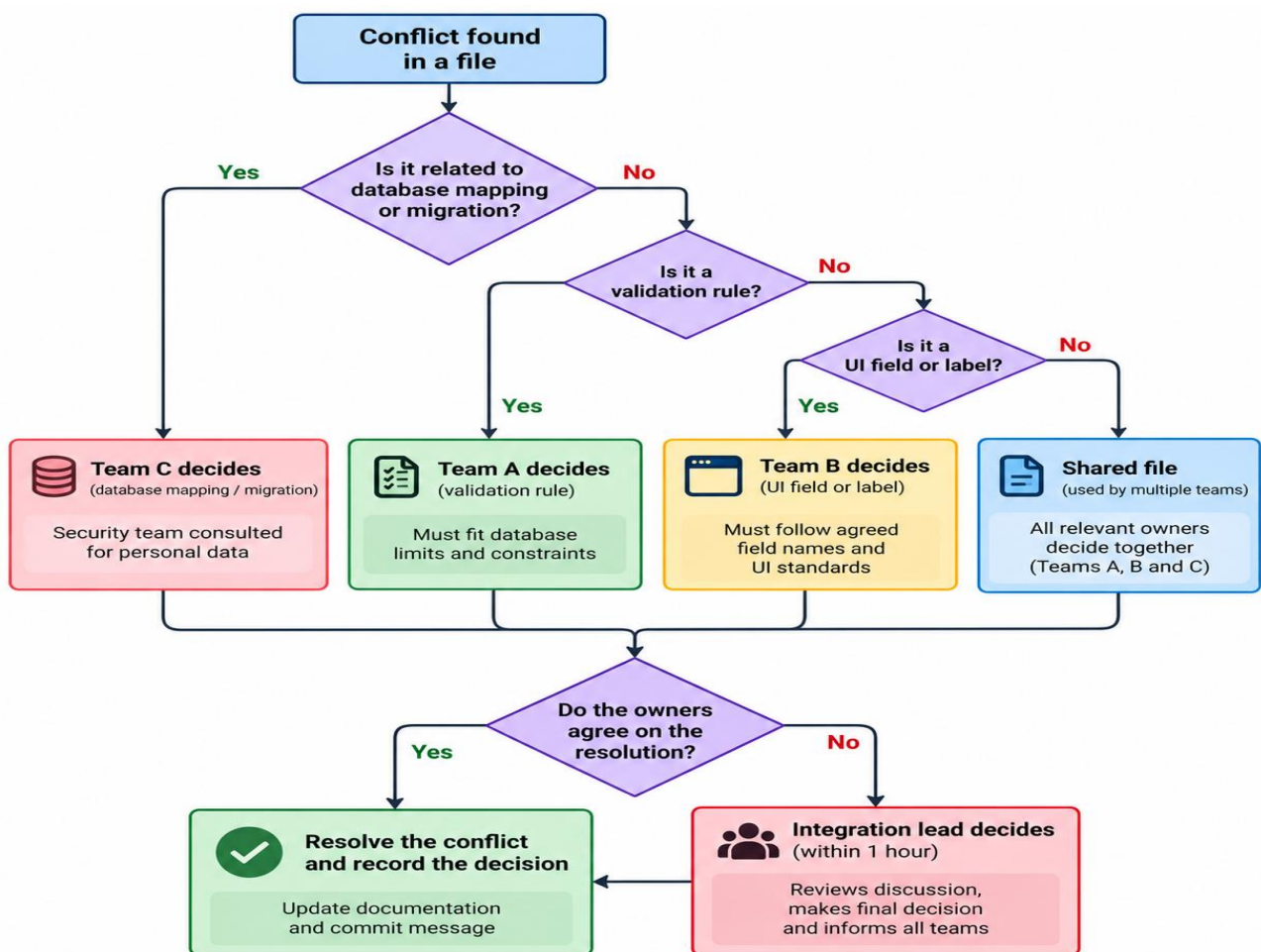


Diagram 1: OWNERSHIP DECISION FLOW

```
git checkout integration/profile-sync
git merge --no-ff team-c/db-mapping      # Team C first
# resolve conflicts, then:
git add <file>
git commit                               # completes the merge
# run tests, then repeat for the next team:
git merge --no-ff team-a/validation
git merge --no-ff team-b/ui-fields

# Made a mess? Start the merge again:
git merge --abort
```

`git merge` is used (not `rebase`) on the shared integration branch because it keeps each team's history intact. Force pushing is not allowed on this branch.

Worked example (illustrative)

After Team C is merged, merging Team A produces a conflict in `profile/models.py`:

```
<<<<<<< HEAD                                (integration branch, with Team C's work)
phone = Column("mobile_no", String(15), nullable=False)
=====
phone = Column(String(10), nullable=False)      # Team A: 10 digits
>>>>>>> team-a/validation
```

Analysis. Team C changed the column name and size; Team A wanted a 10-digit limit. Both are valid, but they belong in different layers: the database only needs to store a number, while the rule that it must have 10 digits belongs to validation.

Resolution. Keep Team C's mapping and size, and keep Team A's rule in `validation.py`:

```
phone = Column("mobile_no", String(15), nullable=False) # Team C's mapping kept
# Team A's 10-digit rule is enforced in profile/validation.py
```

No team's change was lost. The semantic problem (three different names for the same field) is solved by agreeing one name per layer: UI label "Mobile Number", API field `mobile_number`, database column `mobile_no`, and `validation.py` updated to check `mobile_number`.

6.4 Test the Integrated Solution

Key idea: a conflict-free merge does not prove the code works. Testing is how semantic conflicts are caught.

Level	What is checked	How
1. Leftovers	No conflict markers left in any file	<code>git diff --check</code> and search for <code><<<<<<<</code>
2. Build	The code compiles and the pipeline passes	GitLab pipeline on the integration branch
3. Unit tests	Each team's own tests still pass	Run the module's unit tests
4. Integration tests	UI to API to validation to database works end to end	Create, view and edit a test customer profile
5. Intent check	Every team's original feature still works	Checklist: Team A rules, Team B fields, Team C mappings
6. Regression	Nothing else in the platform broke	Full automated suite
7. Data and privacy	Data lands in the right columns; no personal data in logs	Check with test records; Security reviews

```
git diff --check                                # finds leftover markers and whitespace errors
grep -rn "<<<<<<<|>>>>>>>" .                # double-check no markers remain
pytest tests/                                   # backend tests (example)
npm test                                         # frontend tests (example)
```

Tests run **after each team's merge**, so if something breaks the cause is one of only a few changes.

6.5 Document Decisions

Decisions are recorded in a **decision log** kept in the repository (for example `docs/decisions/profile-integration.md`) and referenced in the merge commit message.

Field	Example entry (illustrative)
ID	D-01
File / area	profile/models.py: phone column
Conflict	Team A set length 10; Team C renamed the column to <code>mobile_no</code> with length 15
Options considered	(1) Keep A's size; (2) keep C's size; (3) keep C's mapping, enforce A's rule in validation
Decision	Option 3
Decided by	Owners of Team C and Team A; Team B informed
Reason	The database stores data; the 10-digit rule is a validation rule. Nothing is lost.
Follow-up	Update validation to use <code>mobile_number</code> ; add a test for 10 digits
Date	[date]

```
merge(profile): integrate Teams A, B and C changes

Resolved conflicts in api.py, models.py, ProfileForm.jsx, migrations.
Decisions D-01 to D-05: see docs/decisions/profile-integration.md
Refs: CUST-210
```

6.6 Why This Approach Minimises Risk

Risk (from Section 4)	How the approach reduces it
Lost work (C4)	Owners combine both intentions; "accept one side" is not allowed; the intent check confirms each feature still works
Wrong decision (C3)	The owner of each layer takes part; a tie-breaker exists (integration lead)
Hidden breakage (C2, C9)	Tests run after every merge, including end-to-end and intent checks
Data problems (C5, C7)	The data layer is merged first and settled before other layers; Security reviews personal-data fields
Big late conflicts (C6)	One team at a time keeps each conflict small and the cause obvious
No audit trail (C8)	Every decision is logged with who, why and when
Unsafe history changes	Merge (not rebase) on the shared branch; no force push; <code>git merge --abort</code> available for retries

In one sentence: small steps, the right people deciding, and tests after every step mean mistakes are found early, when they are cheap to fix.

7. Implementation Plan

An integration like this can be done in about **three working days** (an assumption). A prevention plan follows in the second table.

7.1 Integration Plan

Phase	When	Activities	Output
1. Prepare	Day 1 morning	Announce freeze; create integration branch; list each team's changed files; invite owners	Conflict map; owners named
2. Merge Team C	Day 1	Merge data layer; resolve with owners; test; log decisions	Stable data layer
3. Merge Team A	Day 2 morning	Merge validation; resolve; test; log decisions	Validation working on top of data layer
4. Merge Team B	Day 2 afternoon	Merge UI; resolve; test; log decisions	Full integrated module
5. Full testing	Day 3 morning	End-to-end, intent, regression, data and privacy checks	Test report
6. Sign-off and merge	Day 3 afternoon	Three teams and QA sign off; merge request into <code>develop</code> ; end freeze	Integration complete
7. Review	Within one week	Short retrospective: what caused the conflicts, what to change	Lessons and actions

7.2 Prevention Plan

Action	Effect
Agree field names and API contract before coding starts	Prevents semantic conflicts
Merge from <code>develop</code> into feature branches at least daily	Keeps conflicts small and early
Keep branches and merge requests small	Fewer lines can conflict
Keep CODEOWNERS up to date for the module	Clear owners for every file
Run the pipeline on every merge request	Catches breakage immediately
Agree migration numbering rules (for example, one team reserves numbers)	Prevents migration order conflicts

8. Measurable Success Criteria

Targets are illustrative (Assumption A8). Real baselines should be measured and targets adjusted.

#	Metric	How measured	Target
S1	Team changes lost during integration	Intent checklist per team	0
S2	Conflicted files reviewed by their owner	Decision log	100%
S3	Time to complete integration	Plan vs actual	3 working days or less
S4	Automated tests passing on the integrated branch	GitLab pipeline	100%
S5	Critical defects in the profile module in the first 2 weeks	Defect tracker	0
S6	Decisions recorded in the log	Decision log vs conflicts found	100%
S7	Force pushes to shared branches	Platform audit log	0
S8	Conflict markers left in code	<code>git diff --check</code> and pipeline	0
S9	Team sign-off	Sign-off record	3 of 3 teams
S10	Repeat conflicts of the same kind in the next two sprints	Retrospective	50% fewer

9. Recommendations and Lessons Learned

9.1 Recommendations

1. Integrate on a dedicated branch, one team at a time, with the data layer first.
2. Never resolve a conflict by accepting one side blindly; combine both intentions.
3. Let the owner of each file help decide, with a tie-breaker for disagreements.
4. Test after every merge, and always include an end-to-end and an intent check.
5. Record every decision in a log kept with the code.
6. Prevent repeats: agree field names early, sync daily, keep merge requests small.

9.2 Lessons Learned

These come from designing this solution, not from a live project.

- **A clean merge is not a correct merge.** Semantic conflicts only show up through testing.
- **Conflicts are mostly a communication problem.** Git shows the lines; people decide the meaning.
- **Order matters.** Settling the hardest-to-change layer first reduces rework.
- **Small steps are safer.** Merging one team at a time makes problems easy to trace.
- **Writing decisions down saves time later** and satisfies auditors.